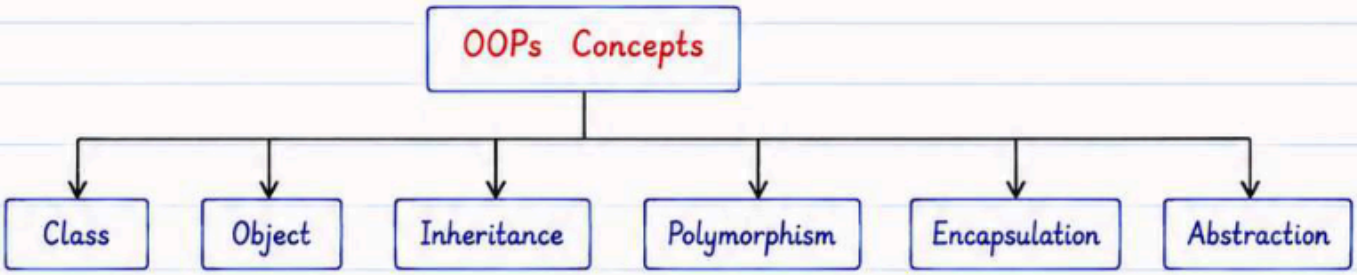
PYTHON OOPS & COLLECTIONS NOTES

1. PYTHON OOPS – INTRODUCTION

★ OOPS stands for Object Oriented Programming.
★ Python is an object oriented language.
★ Everything in Python is an object.
★ OOPs helps to make code modular, reusable and easy to maintain.
★ OOPs is based on the concept of classes and objects.

Key Benefits

- Code reusability
- Better organization
- Easy maintenance
- Data security



2. CLASS AND OBJECT

★ Class is a blueprint or template for creating objects.
★ Object is an instance of a class.
★ A class can have attributes (variables) and methods (functions).

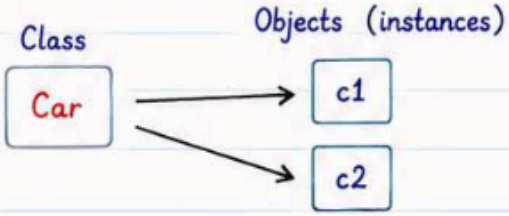
Example :

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color

    def show(self):
        print(self.brand, self.color)

c1 = Car("BMW", "Black")
c2 = Car("Audi", "White")
c1.show()  # Output: BMW Black
c2.show()  # Output: Audi White
```

How it works ?



Car → blueprint
c1, c2 → objects created from the class Car

3. CONSTRUCTOR (__init__)

★ A constructor is a special method used to initialize objects.

★ In Python, __init__() is called automatically when an object is created.

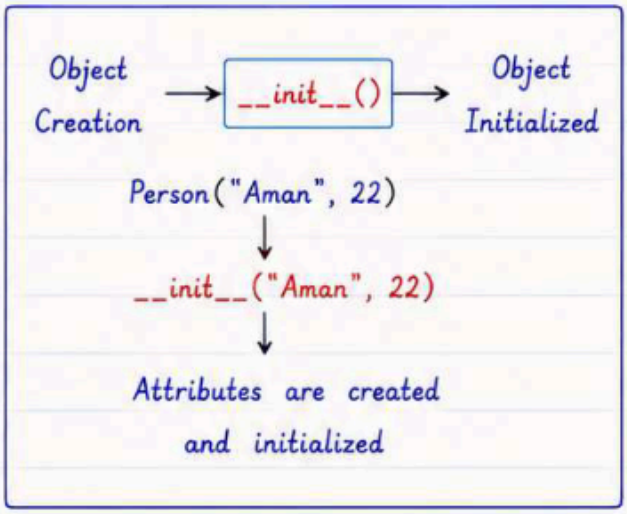
★ It is used to assign initial values to object attributes.

Example :

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Aman", 22)
# Output:
# p1.name = Aman,  p1.age = 22
```

How it works ?



4. INHERITANCE

★ Inheritance allows a class (child) to inherit attributes and methods from another class (parent).

★ It promotes code reusability and represents "is-a" relationship.

★ Python supports single, multiple, multilevel and hierarchical inheritance.

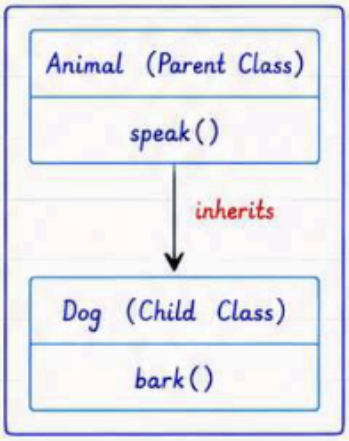
Example (Single Inheritance) :

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):  # Dog inherits from Animal
    def bark(self):
        print("Dog barks")

d = Dog()
d.speak()   # Output: Animal speaks
d.bark()    # Output: Dog barks
```

Diagram :



Key Point :

Child class can use parent class methods and can also have its own methods.

7. ABSTRACTION

★ Abstraction hides the complex implementation and shows only the essential features.

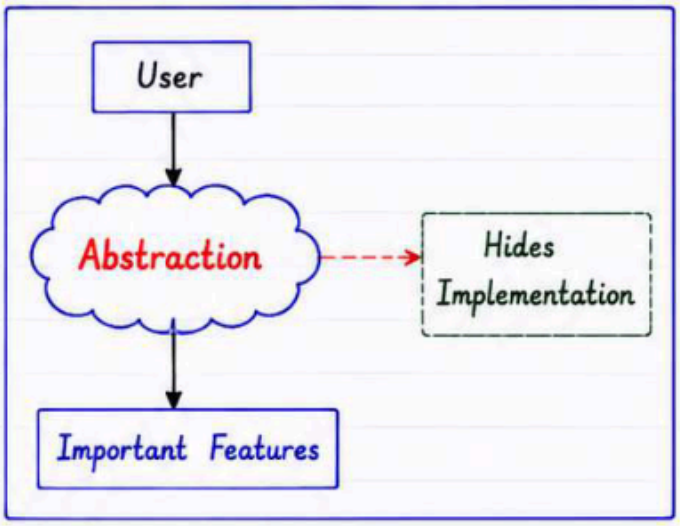
★ In Python, we achieve abstraction using abstract classes and interfaces.

Key Point :
Abstraction focuses on what to do instead of how to do.

Example :

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r * self.r
c = Circle(5)
print(c.area())    # Output: 78.5
```

Abstraction Concept.



8. SPECIAL / MAGIC METHODS

★ Python provides special methods (dunder methods) to customize the behavior of objects.

★ They start and end with double underscore __.

Example :

```
class Book:
    def __init__(self, title, price):
        self.title = title
        self.price = price
    def __str__(self):
        return f"{self.title} - {self.price}"
    def __add__(self, other):
        return self.price + other.price
b1 = Book("Python Basics", 300)
b2 = Book("Java Guide", 200)
print(b1)        # Output: Python Basics - 300
print(b1 + b2)   # Output: 500
```

Common Magic Methods

Method	Purpose
__init__()	Constructor
__str__()	String representation
__len__()	Length of object
__add__()	Addition of objects
__eq__()	Equality check

Summary

- OOPs makes code modular and reusable.
- Classes and objects are the building blocks.
- Python supports all core OOPs concepts.
- Magic methods give more power to objects.

PYTHON OOPS & COLLECTIONS NOTES

1. PYTHON COLLECTIONS – INTRODUCTION

★ Collections are used to store multiple items in a single variable.

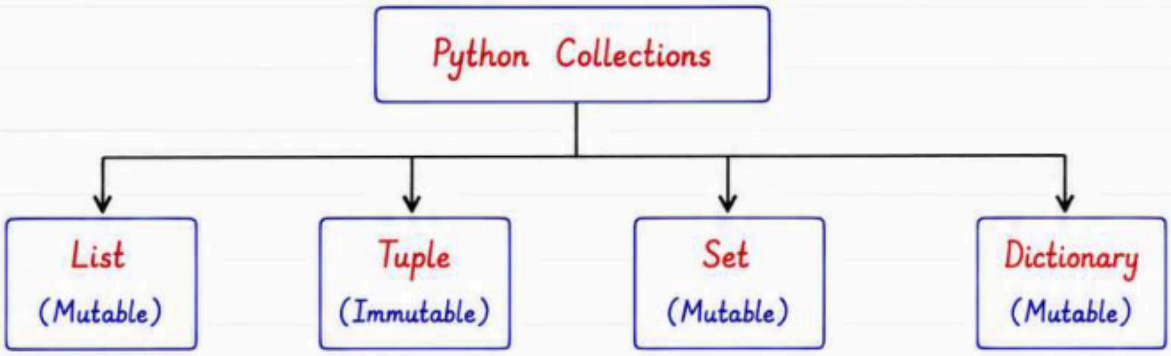
★ Python has four built-in collection datatypes.

★ They are mutable (changeable) except Tuple which is immutable.

★ Choosing the right collection type makes the program efficient.

Why Collections ?

- Store multiple values together
- Easy to access and manage
- Built-in functions & methods
- Improve code readability



2. COLLECTIONS OVERVIEW

Collection Type	Ordered	Mutable	Allows Duplicates	Syntax	Use When
List	Yes	Yes	Yes	[]	Ordered items, changeable, duplicates allowed
Tuple	Yes	No	Yes	()	Ordered items, fixed, duplicates allowed
Set	No	Yes	No	{}	Unique items, membership testing
Dictionary	Yes (3.7+)	Yes	No (Keys)	{key: value}	Key-Value pairs, fast lookups

3. EXAMPLES

```
List       : my_list = [10, 20, 30, 40]
Tuple      : my_tuple = (1, 2, 3, 4)
Set        : my_set = {10, 20, 30}
Dictionary : my_dict = {"name": "Aman", "age": 22}
```

Key Points :

- List, Set, Dictionary are mutable.
- Tuple is immutable.
- Set stores unique items only.
- Dictionary stores data in key-value pair.

4. LIST (Mutable, Ordered)

★ List is used to store multiple items in a single variable.
★ It is ordered, changeable and allows duplicate values.
★ Elements can be of any data type.

Example :

```
my_list = [10, 20, 30, 40]

mixed_list = ["Aman", 22, 5.5, True]

# Accessing elements
print(my_list[0])    # 10
print(my_list[-1])   # 40

# Slicing
print(my_list[1:3])  # [20, 30]
```

Common List Operations

append(x)	–	Add item at the end
insert(i, x)	–	Insert at index i
extend(iterable)	–	Add multiple items
remove(x)	–	Remove first occurrence of x
pop(i)	–	Remove and return item at index i
index(x)	–	Return index of first occurrence
count(x)	–	Count occurrences of x
sort()	–	Sort the list
reverse()	–	Reverse the list
clear()	–	Remove all items

List Methods with Example

Method	Example	Output
append(x)	my_list.append(50)	[10, 20, 30, 40, 50]
insert(i, x)	my_list.insert(1, 15)	[10, 15, 20, 30, 40]
extend([60,70])	my_list.extend([60,70])	[10, 20, 30, 40, 60, 70]
remove(30)	my_list.remove(30)	[10, 20, 40]
pop()	my_list.pop()	40 (returned)
index(20)	my_list.index(20)	1
count(20)	my_list.count(20)	1
sort()	my_list.sort()	[10, 20, 30, 40]
reverse()	my_list.reverse()	[40, 30, 20, 10]
clear()	my_list.clear()	[]

Key Point :

- Lists can be changed
- Lists allow duplicates
- Lists maintain order
- Lists are written using square brackets []

List Comprehension

Compact way to create lists.

Syntax :

```
[expression for item in iterable if condition]
```

Example :

```
squares = [x*x for x in range(1,6)]
print(squares)
# Output: [1, 4, 9, 16, 25]
```

Example with condition :

```
even = [x for x in range(1,11)
        if x % 2 == 0]
print(even)
# Output: [2, 4, 6, 8, 10]
```

PYTHON OOPS & COLLECTIONS NOTES

8. COLLECTION OPERATIONS – QUICK OVERVIEW

Collection	Ordered	Mutable	Allows Duplicates	Access	Use When
List	Yes	Yes	Yes	Index / Slice	Need ordered, mutable collection
Tuple	Yes	No	Yes	Index / Slice	Need ordered, immutable collection
Set	No	Yes	No	No Index	Need unique items (membership testing)
Dictionary	Yes (3.7+)	Yes	No (Keys)	Key	Need key-value mapping (Fast lookup)

9. DICTIONARY METHODS (Detailed)

Example :

```
d = {'name': 'Aman', 'age': 22, 'city': 'Delhi'}
d.get('name')      # Output: Aman
d.keys()           # Output: dict_keys(['name', 'age', 'city'])
d.values()         # Output: dict_values(['Aman', 22, 'Delhi'])
d.items()          # Output: dict_items([('name', 'Aman'),
                                         ('age', 22), ('city', 'Delhi')])
d.update({'age': 23, 'gender': 'M'})
d.pop('city')      # Removes city
d.clear()          # Removes all items
```

Common Dictionary Methods

Method	Description
get(key, default)	Returns value for key, else default
keys()	Returns all keys
values()	Returns all values
items()	Returns key-value pairs
update()	Update dictionary with items
pop(key)	Remove and return value for key
popitem()	Remove and return last inserted pair
clear()	Remove all items from dictionary

10. COMPREHENSIONS

★ Comprehensions provide a concise way to create lists, sets and dictionaries.

★ They improve readability and are generally faster.

List Comprehension

```
[expression for item in iterable if condition]
```

Example :

```
squares = [x*x for x in range(1, 6)]
print(squares)
# Output: [1, 4, 9, 16, 25]
```

Set Comprehension

```
{expression for item in iterable if condition}
```

Example :

```
even_set = {x for x in range(1, 11) if x % 2 == 0}
print(even_set)
# Output: {2, 4, 6, 8, 10}
```

Dictionary Comprehension

```
{key_expr: value_expr for item in iterable if condition}
```

Example :

```
cube_dict = {x: x**3 for x in range(1, 5)}
print(cube_dict)
# Output: {1: 1, 2: 8, 3: 27, 4: 64}
```

Key Takeaways

- List is ordered & mutable.
- Tuple is ordered & immutable.
- Set is unordered & stores unique items.
- Dictionary stores key-value pairs.
- Choose the right collection for the right job!

When to Use What?

